SHIV NADAR UNIVERSITY CHENNAI

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

26AI5701 - ADVANCED DATA STRUCTURES AND ALGORITHMS LABORATORY

Name	Karthikeyan M
Reg. No.	26120004
Experiment No.	6
Date of Expt.	09-10-2026
System No.	
Lab Name	Advanced Data Structures and Algorithms Laboratory

Experiment 6: Tries and Compressed Tries — Array vs. Hash-Map Children

1. Aim

To implement a trie and a compressed trie, each with two child representations — an array of size |Σ| and a hash map (dictionary) — and to compare the two representations on a real search-box autocomplete task.

2. Dataset

- English word-frequency list (333,333 words, one "word count" pair per line): https://www.norvig.com/ngrams/count_1w.txt
- Use the 100,000 most frequent words; skip words with characters outside a–z.

3. What is Expected

Trie

- Operations: insert(word, freq), search(word), startsWith(prefix).
- Two versions of the node's children: (i) array of size |Σ|, (ii) hash map / dictionary holding only the children that exist.

Compressed Trie

- Collapse chains of single-child nodes (that do not end a word) into one edge labelled by a string.
- Insertion must split an edge when a new word diverges part-way along it.
- Same two child representations as the trie.

Real-life Demo — Search-box Autocomplete

- autocomplete(prefix, k): the k most frequent words starting with prefix (e.g. "progr" → program, programs, …).

- Show the top-5 suggestions for at least five prefixes.

Measurements (for all four structures)

- Number of nodes and memory used.
- Build time, search time (word present / absent), autocomplete time.
- Repeat the memory comparison on random strings over larger alphabets: $|\Sigma| = 4$ (DNA), 26, 62, 95.
- Count how many array slots are actually used at each depth of the trie.

Comparison

- Prepare a comparison of the two representations (array of size $|\Sigma|$ vs. hash map) using your measurements: lookup cost, memory per node, effect of $|\Sigma|$, speed, alphabetical ordering, and when you would choose each.
- Which of the four structures would you deploy for autocomplete, and why?

4. Program

Attach the complete, commented source code and its output to this record.

Complete commented source code is attached in the appendix and supplied as lab6/trie.py and lab6/compressed_trie.py. No external Python packages are needed.

Run from the repository folder after downloading count_1w.txt:

```
python3 lab6/trie.py count_1w.txt
```

The program prints JSON. The tables below present its measured output, rounded for readability. All four versions returned the same suggestions.

5. Observations

Present your measurements as tables and plots of your own design.

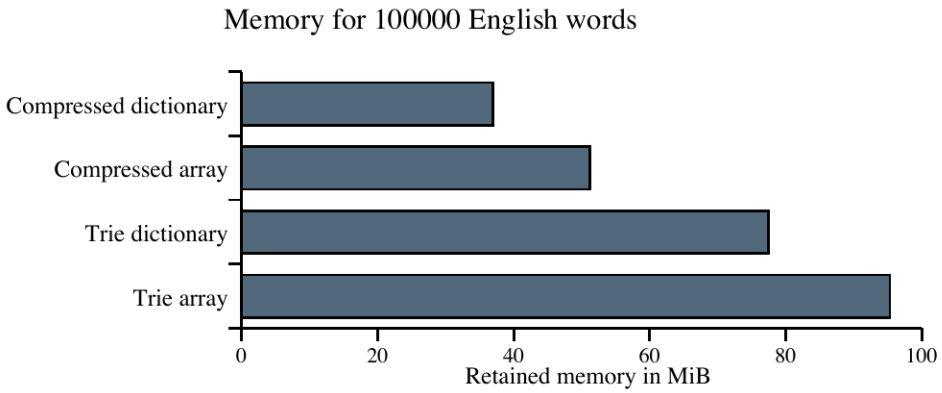
Measurements on English words

Python 3.9.6; macOS-27.0.1-arm64-arm-64bit. The 100000 most frequent words containing only a-z were inserted once. Search times are medians of five batches of 1000 queries. Absent queries append zzzzzz to present words. Autocomplete times are medians of five batches over the five prefixes below.

Memory is a retained-object estimate using sys.getsizeof, counting each reachable object once, including nodes, child containers, edge labels, frequencies and trie metadata. It excludes the input list and interpreter allocator overhead. Build time excludes loading and validation.

Structure	Nodes	Memory MiB	Bytes per node	Build s
Trie array	233,569	95.34	428.0	0.368
Trie dictionary	233,569	77.50	347.9	0.267
Compressed array	115,733	51.18	463.7	0.301
Compressed dictionary	115,733	36.93	334.6	0.255

Structure	Present us/query	Absent us/query	Autocomplete us/query
Trie array	2.07	2.16	171.78
Trie dictionary	1.91	2.07	96.79
Compressed array	1.59	1.68	89.98
Compressed dictionary	1.56	1.74	57.63



Average bytes per node includes stored strings and frequencies. Timings describe this Python implementation and workload; they can vary between runs.

Autocomplete output

Top five words are ordered by decreasing frequency. Counts are the dataset frequencies. Equal counts use alphabetical order.

Prefix	Top five suggestions and counts
progr	program (306,686,983) programs (142,498,232) programme (57,212,448) programming (50,630,393) progress (47,674,782)
comp	company (324,272,258) computer (224,177,047) compare (183,202,885) complete (149,312,237) companies (118,106,080)
data	data (406,908,328) database (103,731,642) databases (21,617,803) datasheet (3,680,819) dataset (3,064,674)
art	art (214,702,696) article (181,969,355) articles (151,531,225) arts (106,466,033) artist (67,421,654)
uni	university (311,373,936) united (299,280,163) unit (93,714,361) union (72,603,888) unique (63,116,899)

Memory on random strings

Each alphabet uses 10000 random strings of length 12, with seed 42. The identical sample is inserted into all four versions. Alphabet 4 is ACGT; 26 is lowercase a-z; 62 adds uppercase letters and digits; 95 is printable ASCII including space. Repeated strings overwrite their frequency.

Alphabet size	Trie array	Trie dictionary	Compressed array	Compressed dictionary
4	14.11	20.98	4.27	4.89
26	38.98	34.60	5.60	3.83
62	69.45	36.42	9.38	3.91
95	97.99	37.47	12.45	3.88

All memory entries are MiB. Arrays use less memory for the small DNA alphabet; dictionaries become more economical as the alphabet grows.

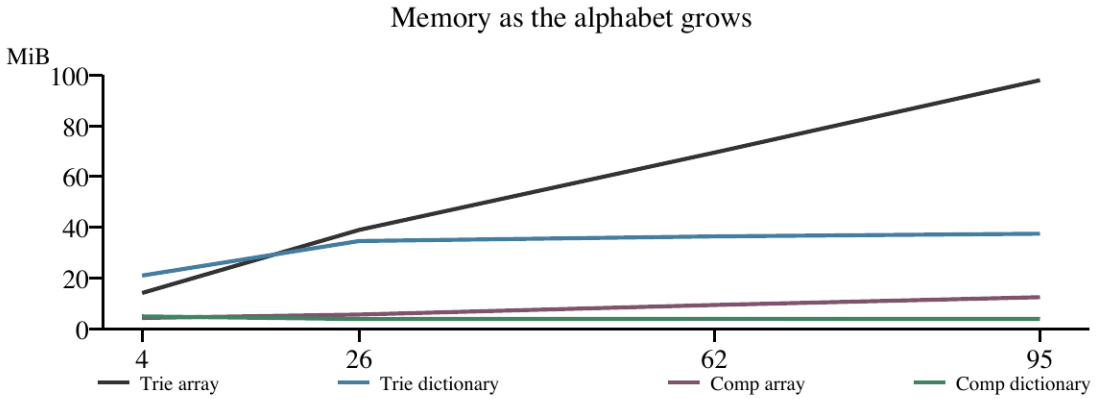
Array slots used at each depth

Depth counts characters from the root, including compressed edge lengths. Slots = nodes x 26; used slots contain a child. A missing compressed depth has no nodes. T means ordinary trie; C means compressed trie.

Depth	T nodes	T slots	T used	T used %	C nodes	C slots	C used	C used %
0	1	26	26	100.00	1	26	26	100.00
1	26	676	676	100.00	26	676	676	100.00
2	676	17,576	8,899	50.63	676	17,576	8,899	50.63
3	8,899	231,374	21,433	9.26	8,080	210,080	20,614	9.81
4	21,433	557,258	35,409	6.35	12,982	337,532	26,958	7.99
5	35,409	920,634	40,941	4.45	16,029	416,754	21,561	5.17
6	40,941	1,064,466	37,932	3.56	17,014	442,364	14,005	3.17
7	37,932	986,232	30,651	3.11	16,502	429,052	9,221	2.15
8	30,651	796,926	22,168	2.78	14,243	370,318	5,760	1.56
9	22,168	576,368	14,804	2.57	11,103	288,678	3,739	1.30
10	14,804	384,904	9,031	2.35	7,965	207,090	2,192	1.06
11	9,031	234,806	5,302	2.26	4,863	126,438	1,134	0.90
12	5,302	137,852	2,934	2.13	2,889	75,114	521	0.69
13	2,934	76,284	1,525	2.00	1,659	43,134	250	0.58
14	1,525	39,650	823	2.08	812	21,112	110	0.52
15	823	21,398	398	1.86	464	12,064	39	0.32
16	398	10,348	236	2.28	182	4,732	20	0.42
17	236	6,136	128	2.09	114	2,964	6	0.20
18	128	3,328	82	2.46	47	1,222	1	0.08
19	82	2,132	52	2.44	30	780	0	0.00
20	52	1,352	33	2.44	19	494	0	0.00
21	33	858	27	3.15	6	156	0	0.00
22	27	702	18	2.56	9	234	0	0.00
23	18	468	14	2.99	4	104	0	0.00
24	14	364	8	2.20	6	156	0	0.00
25	8	208	4	1.92	4	104	0	0.00
26	4	104	3	2.88	1	26	0	0.00
27	3	78	3	3.85	0	0	0	-
28	3	78	3	3.85	0	0	0	-
29	3	78	2	2.56	1	26	0	0.00
30	2	52	2	3.85	0	0	0	-
31	2	52	1	1.92	1	26	0	0.00
32	1	26	0	0.00	1	26	0	0.00

Total ordinary slots: 6,072,794; used: 233,568 (3.85%). Total compressed slots: 3,009,058; used: 115,732 (3.85%). Leaves allocate arrays too, although they use no child slots.

Comparison of child representations



Lookup cost: array indexing is O(1); dictionary lookup is expected O(1). Both search a word of length L in O(L). This array implementation also uses a character-to-index dictionary, so Python overhead affects timings.

Memory per node: arrays reserve a slot for every alphabet character, including missing children. Dictionaries store existing children. For English, average retained bytes per ordinary node were 428.0 for arrays and 347.9 for dictionaries. At alphabet size 95, ordinary arrays used 97.99 MiB versus 37.47 MiB for dictionaries. At size 4, arrays were smaller.

Speed: compressed dictionaries built fastest and had the lowest present search and autocomplete times in this run. Compressed arrays were slightly faster for absent searches. Compression reduces node visits; dictionary traversal also avoids scanning empty array slots.

Ordering and choice: traversing arrays in the alphabet order gives alphabetical order when the alphabet is sorted. Dictionary keys need sorting for that order. Choose arrays for a small fixed alphabet or dense branching; choose dictionaries for sparse nodes or larger alphabets.

For autocomplete, I would choose the compressed dictionary trie. It used 36.93 MiB and averaged 57.63 us per query for these prefixes. Autocomplete enumerates the matching subtree and sorts M matches in O(M log M), so broad prefixes can take longer; no cached suggestions are stored.

6. Result

Implemented and verified ordinary and compressed tries with array and dictionary children. All four versions found the inserted words and agreed with directly filtered top-five suggestions. Compression reduced nodes from 233569 to 115733. The compressed dictionary version gave the smallest English-word memory estimate and fastest autocomplete in this experiment.

Source code trie.py

```
"""Ordinary trie and the measurements for the lab record.

Run: python3 lab6/trie.py count_1w.txt
The dataset is available at https://www.norvig.com/ngrams/count_1w.txt.
"""

import gc
import json
import platform
import random
import statistics
import string
import sys
from time import perf_counter


class TrieNode:
    def __init__(self, alphabet, use_array):
        self.children = [None] * len(alphabet) if use_array else {}
        self.label = ""  # Compressed tries store an edge label here.
        self.freq = None  # None means that no complete word ends here.


class Trie:
    def __init__(self, alphabet=string.ascii_lowercase, use_array=False):
        self.alphabet = alphabet
        self.use_array = use_array
        self.index = {char: i for i, char in enumerate(alphabet)}
        self.root = self.new_node()

    def new_node(self):
        return TrieNode(self.alphabet, self.use_array)

    def get_child(self, node, char):
        if self.use_array:
            return node.children[self.index[char]]
        return node.children.get(char)

    def set_child(self, node, char, child):
        key = self.index[char] if self.use_array else char
        node.children[key] = child

    def child_nodes(self, node):
        children = node.children if self.use_array else node.children.values()
        return [child for child in children if child is not None]

    def insert(self, word, freq):
        if any(char not in self.index for char in word):
            raise ValueError("Word contains a character outside the alphabet")
        node = self.root
        for char in word:
            child = self.get_child(node, char)
            if child is None:
                child = self.new_node()
                child.label = char
                self.set_child(node, char, child)
            node = child
        node.freq = freq
```

Source code trie.py continued

```
    def find_prefix(self, prefix):
        node = self.root
        word = ""
        while prefix:
            if prefix[0] not in self.index:
                return None, ""
            child = self.get_child(node, prefix[0])
            if child is None:
                return None, ""
            # A prefix may finish inside a compressed edge.
            if child.label.startswith(prefix):
                return child, word + child.label
            if not prefix.startswith(child.label):
                return None, ""
            prefix = prefix[len(child.label):]
            word += child.label
            node = child
        return node, word

    def search(self, word):
        node, matched = self.find_prefix(word)
        return node is not None and matched == word and node.freq is not None

    def startsWith(self, prefix):
        node, word = self.find_prefix(prefix)
        return node is not None

    def autocomplete(self, prefix, k):
        node, word = self.find_prefix(prefix)
        if node is None or k <= 0:
            return []
        matches = []
        stack = [(node, word)]
        while stack:
            node, word = stack.pop()
            if node.freq is not None:
                matches.append((word, node.freq))
            for child in self.child_nodes(node):
                stack.append((child, word + child.label))
        # Higher frequency first; alphabetical order breaks ties.
        matches.sort(key=lambda item: (-item[1], item[0]))
        return matches[:k]

    def measurements(self):
        """Count nodes, retained Python object bytes and slots at each depth."""
        seen = set()
        memory = 0

        def count_object(obj):
            nonlocal memory
            if id(obj) not in seen:
                seen.add(id(obj))
                memory += sys.getsizeof(obj)

        count_object(self)
        count_object(self.__dict__)
        count_object(self.alphabet)
```

Source code trie.py continued

```
        count_object(self.use_array)
        count_object(self.index)
        for char, index in self.index.items():
            count_object(char)
            count_object(index)
        count_object(None)
        depths = {}
        nodes = 0
        stack = [(self.root, 0)]
        while stack:
            node, depth = stack.pop()
            nodes += 1
            for obj in (node, node.__dict__, node.children, node.label, node.freq):
                count_object(obj)
            if not self.use_array:
                for char in node.children:
                    count_object(char)
            children = self.child_nodes(node)
            if depth not in depths:
                depths[depth] = [0, 0]
            depths[depth][0] += 1
            depths[depth][1] += len(children)
            for child in children:
                stack.append((child, depth + len(child.label)))
        return nodes, memory / 1024**2, depths


def load_words(filename):
    words = []
    with open(filename) as file:
        for line in file:
            word, freq = line.split()
            if all(char in string.ascii_lowercase for char in word):
                words.append((word, int(freq)))
    words.sort(key=lambda item: -item[1])
    if len(words) < 100000:
        raise ValueError("Dataset must contain at least 100000 valid words")
    return words[:100000]


def time_queries(operation, queries, repeats=5):
    times = []
    for repeat in range(repeats):
        start = perf_counter()
        for query in queries:
            operation(query)
        times.append((perf_counter() - start) * 1000000 / len(queries))
    return statistics.median(times)


def run(filename):
    # Import here so compressed_trie.py can reuse Trie without a circular import.
    from compressed_trie import CompressedTrie

    words = load_words(filename)
    rng = random.Random(42)
    present = [word for word, freq in rng.sample(words, 1000)]
    absent = [word + "zzzzzz" for word in present]
```

Source code trie.py continued

```
    prefixes = ["progr", "comp", "data", "art", "uni"]
    versions = [("Trie array", Trie, True), ("Trie dictionary", Trie, False),
                ("Compressed array", CompressedTrie, True),
                ("Compressed dictionary", CompressedTrie, False)]
    result = {"python": platform.python_version(), "system": platform.platform(),
              "word_count": len(words), "english": [], "random": [],
              "slots": {}, "suggestions": {}}
    for name, structure, use_array in versions:
        gc.collect()
        trie = structure(use_array=use_array)
        start = perf_counter()
        for word, freq in words:
            trie.insert(word, freq)
        build = perf_counter() - start
        assert all(trie.search(word) for word, freq in words)
        assert not any(trie.search(word) for word in absent)
        suggestions = {}
        for prefix in prefixes:
            expected = sorted([item for item in words if item[0].startswith(prefix)],
                              key=lambda item: (-item[1], item[0]))[:5]
            suggestions[prefix] = trie.autocomplete(prefix, 5)
            assert suggestions[prefix] == expected
        nodes, memory, depths = trie.measurements()
        result["english"].append({"name": name, "nodes": nodes, "memory": memory,
                                  "build": build,
                                  "present": time_queries(trie.search, present),
                                  "absent": time_queries(trie.search, absent),
                                  "autocomplete": time_queries(
                                      lambda p: trie.autocomplete(p, 5), prefixes)})
        if use_array:
            result["slots"][name] = depths
        result["suggestions"][name] = suggestions
        del trie

    for alphabet in ["ACGT", string.ascii_lowercase,
                     string.ascii_letters + string.digits,
                     "".join(chr(i) for i in range(32, 127))]:
        # The same 10000 length-12 strings go into all four versions.
        rng = random.Random(42)
        samples = ["".join(rng.choices(alphabet, k=12)) for i in range(10000)]
        for name, structure, use_array in versions:
            gc.collect()
            trie = structure(alphabet, use_array)
            for word in samples:
                trie.insert(word, 1)
            nodes, memory, depths = trie.measurements()
            result["random"].append({"alphabet": len(alphabet), "name": name,
                                     "nodes": nodes, "memory": memory})
            del trie
    print(json.dumps(result, indent=2))


if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "count_1w.txt")
```

Source code compressed_trie.py

```
"""Compressed trie: reuse traversal and measurement methods from Trie."""

from trie import Trie


class CompressedTrie(Trie):
    def insert(self, word, freq):
        if any(char not in self.index for char in word):
            raise ValueError("Word contains a character outside the alphabet")
        node = self.root
        while word:
            child = self.get_child(node, word[0])
            if child is None:
                child = self.new_node()
                child.label = word
                child.freq = freq
                self.set_child(node, word[0], child)
                return

            # Find the common part of the new word and existing edge.
            common = 0
            while (common < len(word) and common < len(child.label)
                   and word[common] == child.label[common]):
                common += 1
            if common == len(child.label):
                word = word[common:]
                node = child
                continue

            # Split the old edge at the point where the words diverge.
            middle = self.new_node()
            middle.label = child.label[:common]
            self.set_child(node, word[0], middle)
            child.label = child.label[common:]
            self.set_child(middle, child.label[0], child)
            word = word[common:]
            if word:
                leaf = self.new_node()
                leaf.label = word
                leaf.freq = freq
                self.set_child(middle, word[0], leaf)
            else:
                middle.freq = freq  # The new word ends inside the old edge.
            return
        node.freq = freq  # Also handles duplicate words and the empty word.


if __name__ == "__main__":
    trie = CompressedTrie()
    for word, freq in [("program", 50), ("programs", 20), ("progress", 30)]:
        trie.insert(word, freq)
    print("Search program:", trie.search("program"))
    print("Prefix progr:", trie.startsWith("progr"))
    print("Suggestions:", trie.autocomplete("progr", 5))
```